

# CS-317 Operating Systems



Lecture 8



# Threads

Book 1 Chapter 4

## In the last lecture...

- ▶ We came to know what a thread is.
- ▶ We came to know the benefits that threads provide us. 
- ▶ We looked at scenarios where the use of threads leads to better applications.

# Activities that only happen at the process level

- ▶ In an OS that supports threads, scheduling and dispatching is done on thread basis: hence, most of the state information dealing with execution is maintained in TCBs.
- ▶ **Suspension** involves swapping the address space of one process out of main memory to make room for the address space of another process. Because all threads in a process share the same address space, all threads are suspended at the same time.
- ▶ **Termination** of a process terminates all threads within that process.



# Thread states

Important points in the life span of a thread

# Thread states



Spawn



Block



Unblock



Finish

## Pause the video!

Write your answer in a word file. Save it as <my\_roll\_no>\_lecture\_8. If your roll no is 500, the file will be named 500\_lecture\_8. Do not add <> signs to the filename.

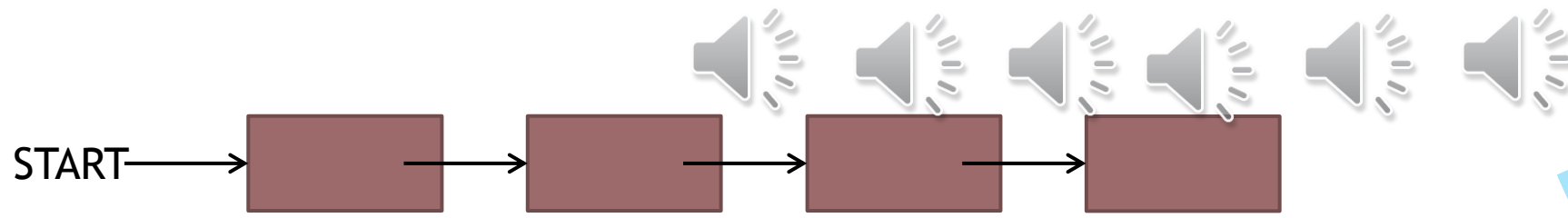
Task# 1: For each thread state, name the equivalent process state from the 7-state process model.

# Thread synchronization

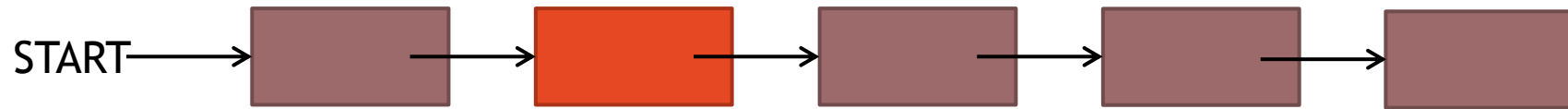
Enforcing order among threads

# Thread synchronization

- ▶ Synchronization means making things happen in a particular chosen order.
- ▶ All the threads of a process share the same address space and other resources, such as open files.
- ▶ If one thread changes data, data appears altered to all the threads in the process.
- ▶ It is therefore necessary to synchronize the activities of the various threads so that they do not interfere with each other or corrupt data structures.



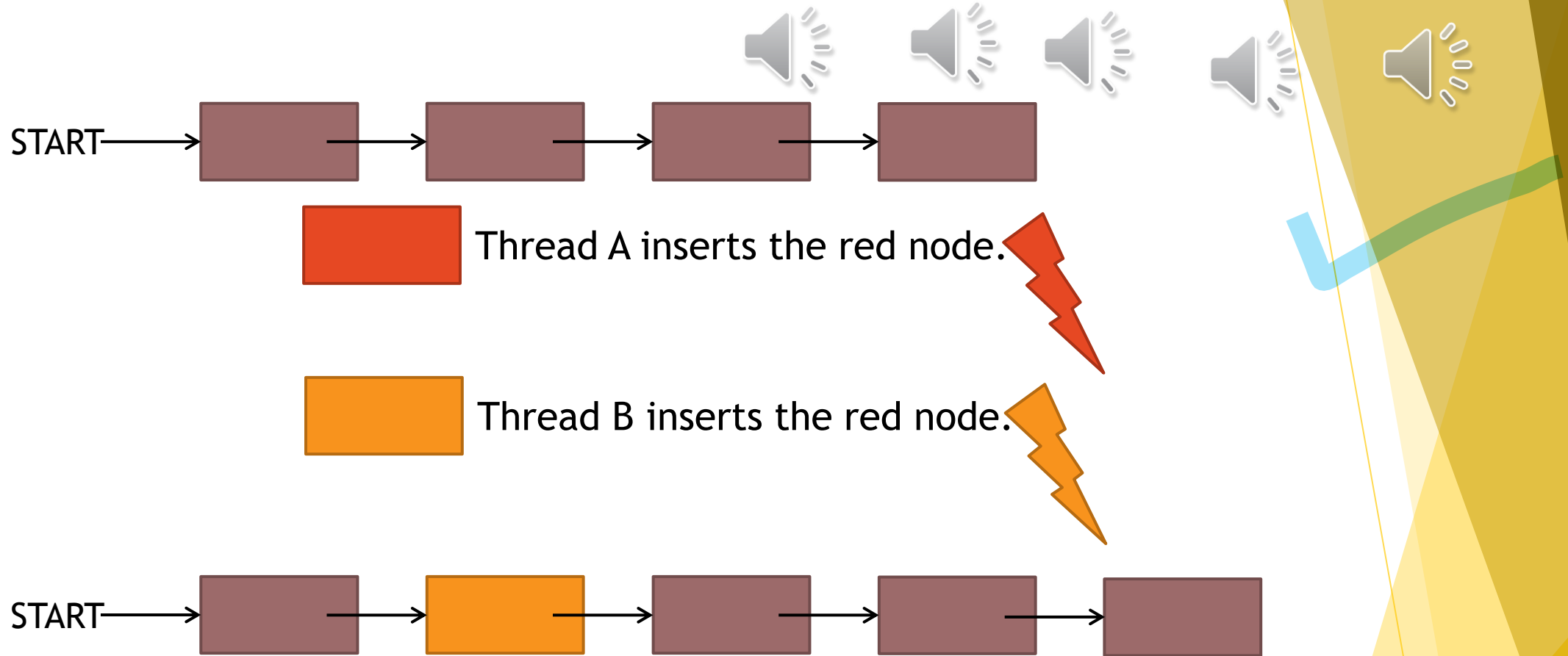
 Thread A inserts the red node.



 Thread B inserts the orange node.



THE INSERTION WE WANT



**UNSYNCHRONIZED INSERTION: BOTH THREADS COME TOGETHER  
ONE NODE IS LOST**

# Remote Procedure Calls



RPCs and Multithreading

# Remote Procedure Call (RPC)

- ▶ A Remote Procedure Call (RPC) is an interaction between two programs that are running on two different machines.
- ▶ One program calls the other in the same way as one function calls another function.
- ▶ The called program returns a result to the calling program in the same way as a called function returns a value to the calling function.
- ▶ RPCs are often used in client/server applications.



## Pause the video!

Write your answer in a word file previously saved as <my\_roll\_no>\_lecture\_8.

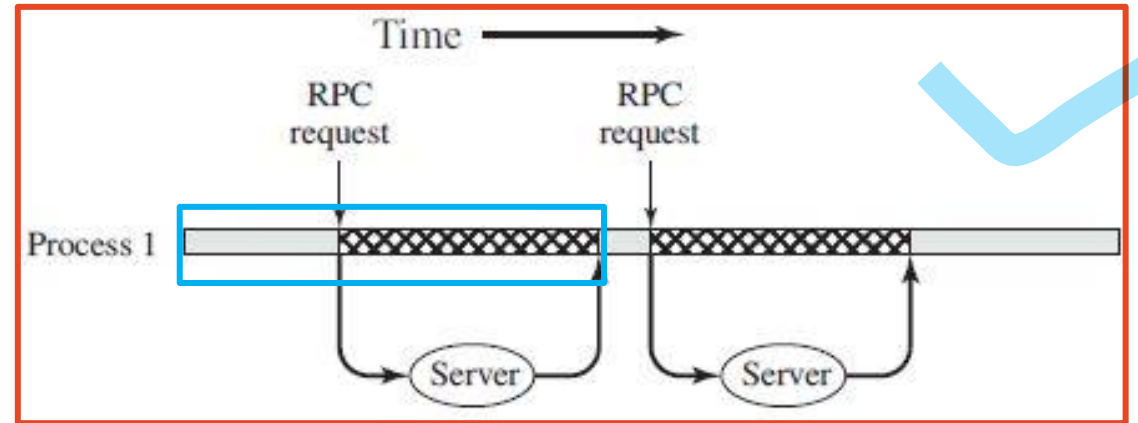
Task# 2: Which of these can be RPCs?

1. Microsoft Word opening a file stored in USB flash drive.
2.  Microsoft Word opening a file from Dropbox.
3. Google Chrome loading a webpage.
4. Downloading OS lectures from Google Drive.

# The role of threads in RPC

## Single-threaded approach:

- Process 1 starts executing, makes the first RPC and blocks.

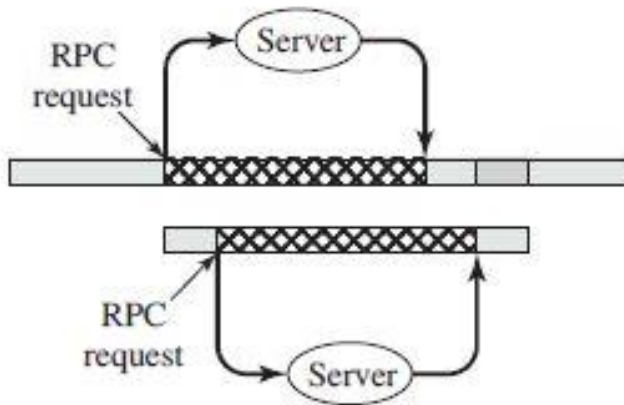


(a) RPC using single thread



Thread A (Process 1)

Thread B (Process 1)



(b) RPC using one thread per server (on a uniprocessor)



Blocked, waiting for response to RPC



Blocked, waiting for processor, which is in use by Thread B



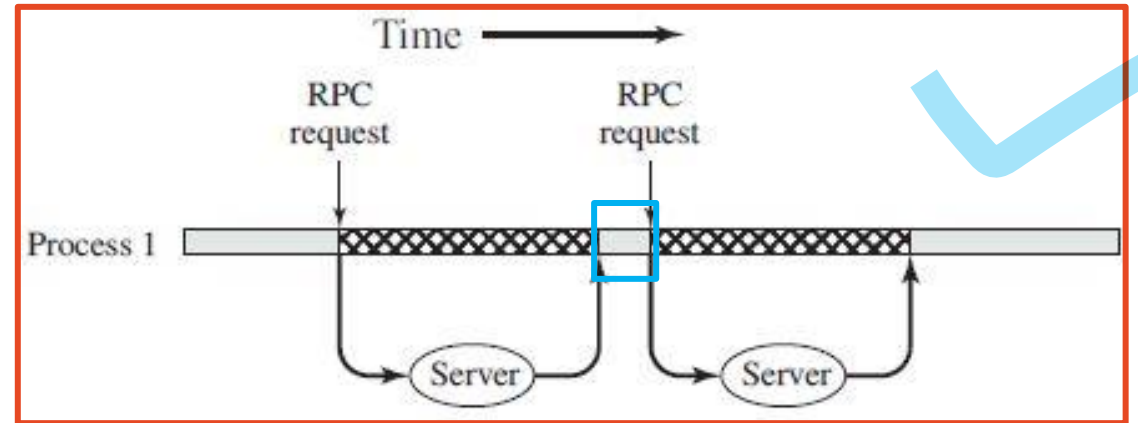
Running

**Figure 4.3** Remote Procedure Call (RPC) Using Threads

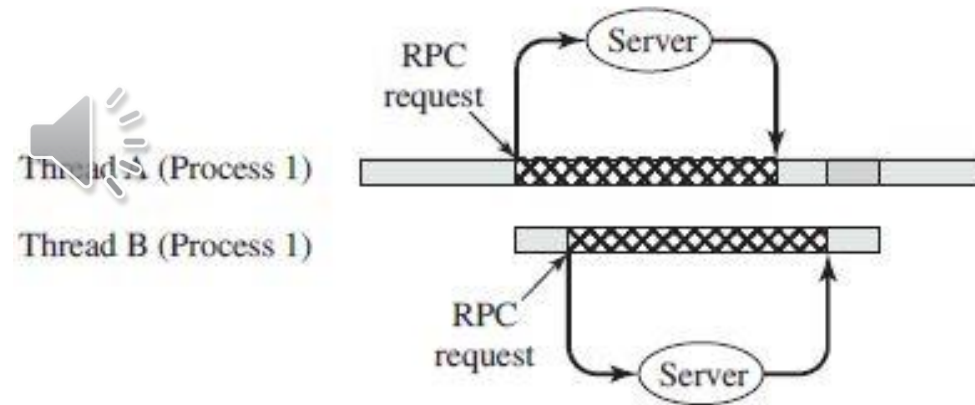
# The role of threads in RPC

## Single-threaded approach:

- ▶ Process 1 starts executing, makes the first RPC and blocks.
- ▶ When the server responds, Process 1 unblocks and resumes.



(a) RPC using single thread



(b) RPC using one thread per server (on a uniprocessor)

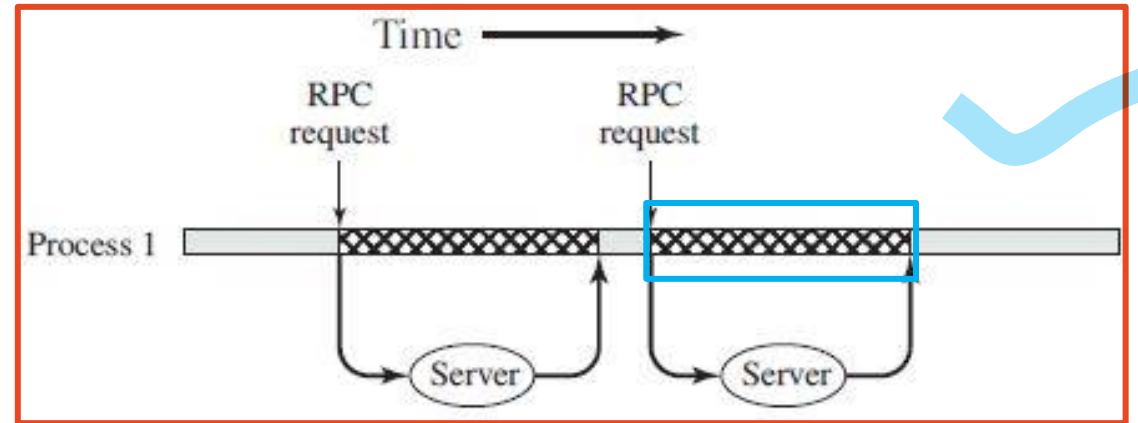
- Blocked, waiting for response to RPC
- Blocked, waiting for processor, which is in use by Thread B
- Running

**Figure 4.3** Remote Procedure Call (RPC) Using Threads

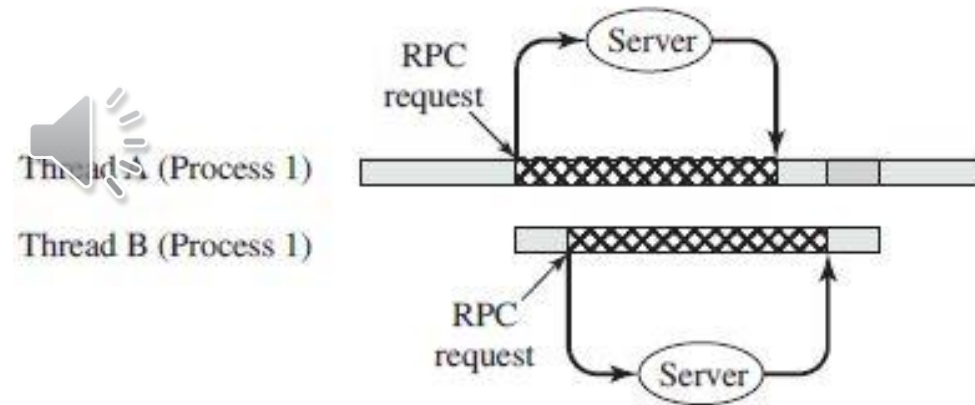
# The role of threads in RPC

## Single-threaded approach:

- ▶ Process 1 starts executing, makes the first RPC and blocks.
- ▶ When the server responds, Process 1 unblocks and resumes.
- ▶ Then it makes the second RPC and blocks.



(a) RPC using single thread



(b) RPC using one thread per server (on a uniprocessor)

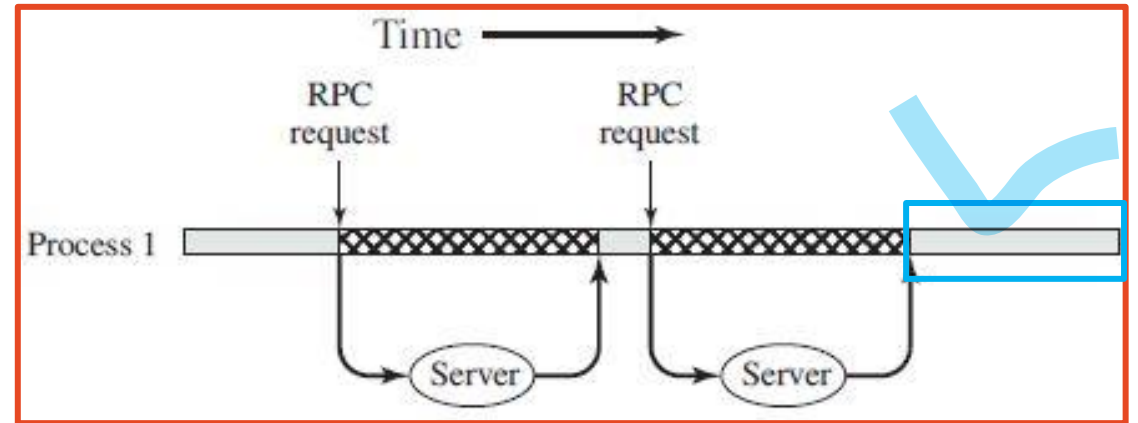
- Blocked, waiting for response to RPC
- Blocked, waiting for processor, which is in use by Thread B
- Running

**Figure 4.3** Remote Procedure Call (RPC) Using Threads

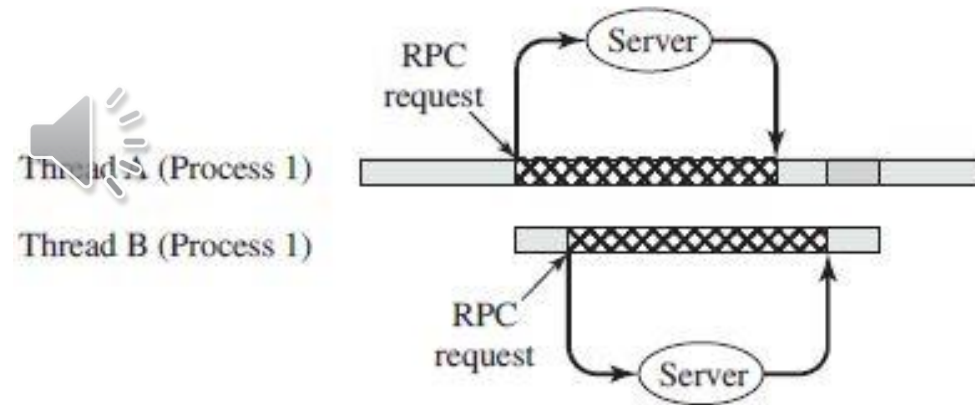
# The role of threads in RPC

## Single-threaded approach:

- ▶ Process 1 starts executing, makes the first RPC and blocks.
- ▶ When the server responds, Process 1 unblocks and resumes.
- ▶ Then it makes the second RPC and blocks.
- ▶ When the second server responds, Process 1 unblocks and resumes until it goes to completion



(a) RPC using single thread



(b) RPC using one thread per server (on a uniprocessor)

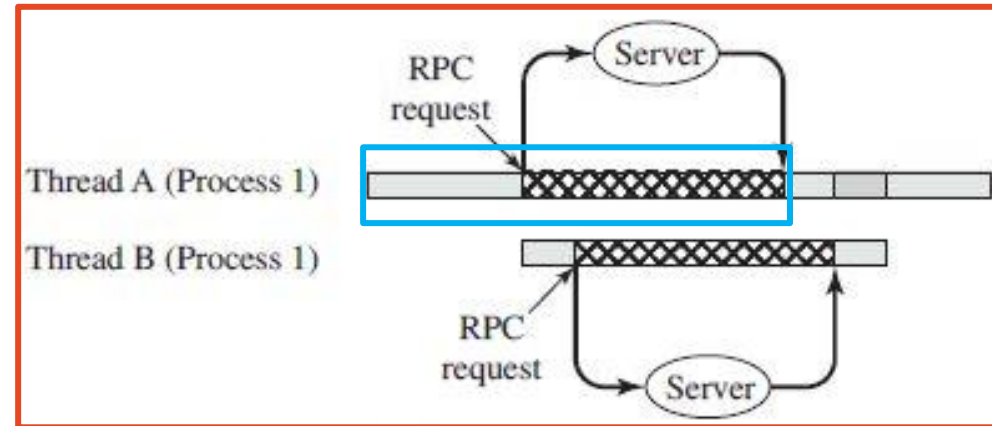
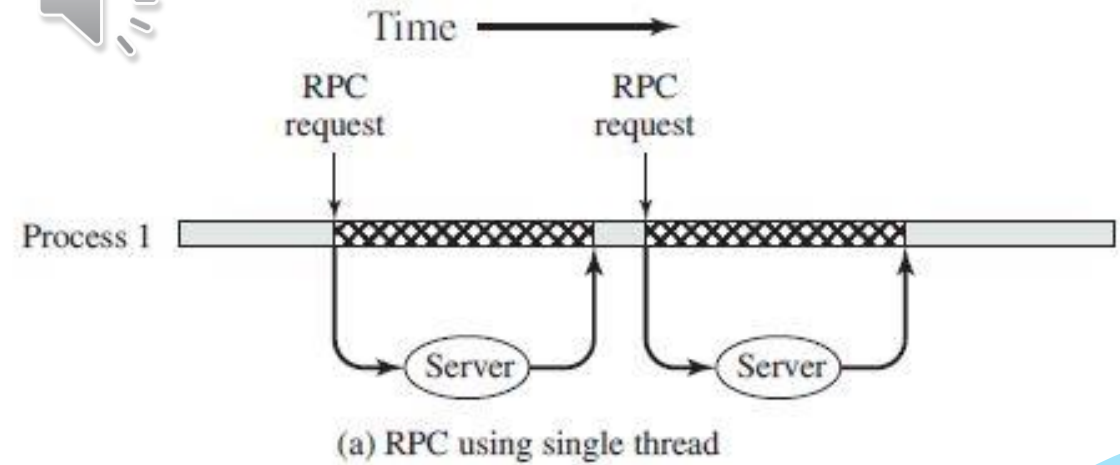
- Blocked, waiting for response to RPC
- Blocked, waiting for processor, which is in use by Thread B
- Running

**Figure 4.3** Remote Procedure Call (RPC) Using Threads

# The role of threads in RPC

## Multithreaded approach:

- Inside Process 1, Thread A starts executing. It makes the first RPC and blocks.



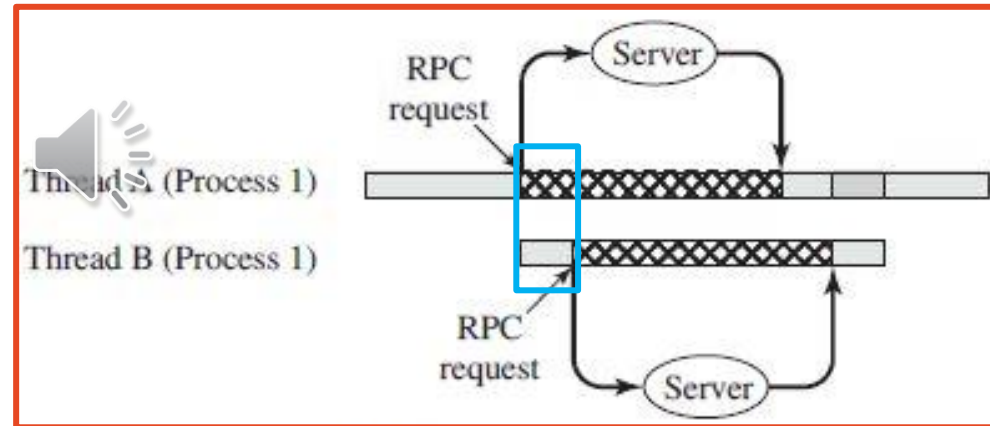
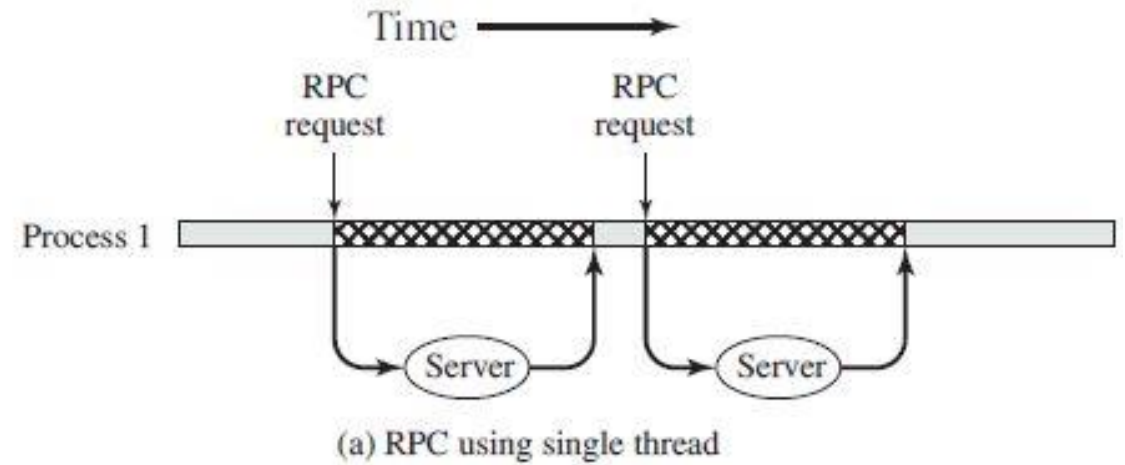
-  Blocked, waiting for response to RPC
-  Blocked, waiting for processor, which is in use by Thread B
-  Running

**Figure 4.3** Remote Procedure Call (RPC) Using Threads

# The role of threads in RPC

## Multithreaded approach:

- ▶ Inside Process 1, Thread A starts executing. It makes the first RPC and blocks.
- ▶ Thread B starts executing when Thread A blocks.



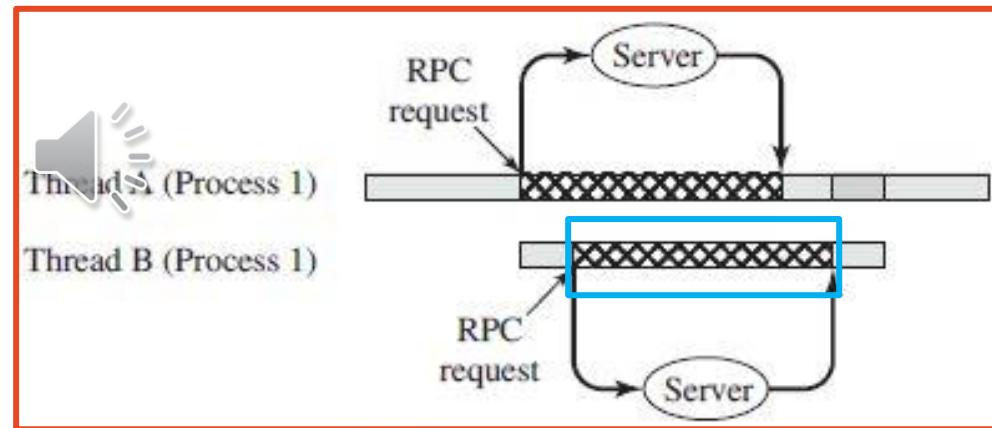
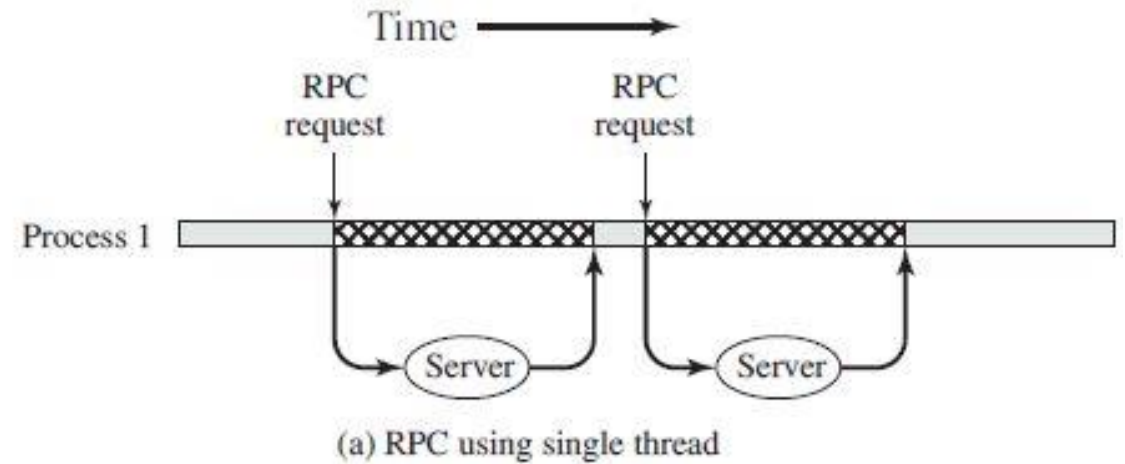
- Blocked, waiting for response to RPC
- Blocked, waiting for processor, which is in use by Thread B
- Running

**Figure 4.3** Remote Procedure Call (RPC) Using Threads

# The role of threads in RPC

## Multithreaded approach:

- ▶ Inside Process 1, Thread A starts executing. It makes the first RPC and blocks.
- ▶ Thread B starts executing when Thread A blocks.
- ▶ Thread B makes the second RPC and blocks.



(b) RPC using one thread per server (on a uniprocessor)

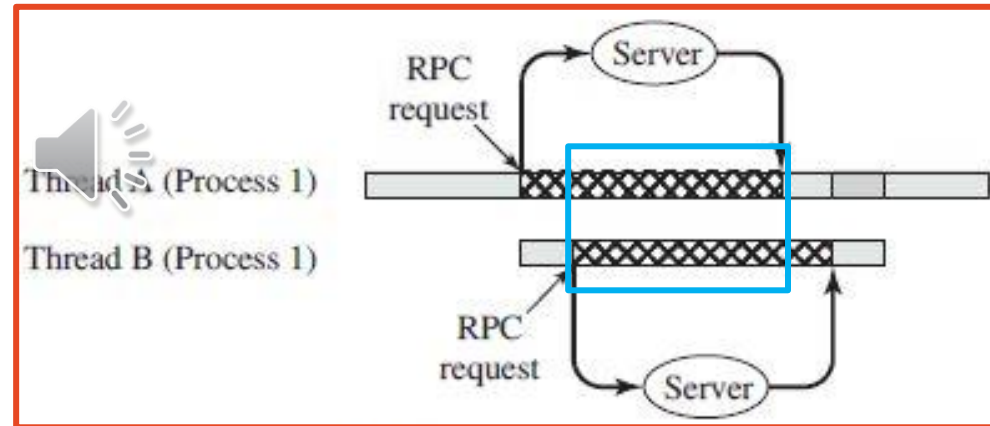
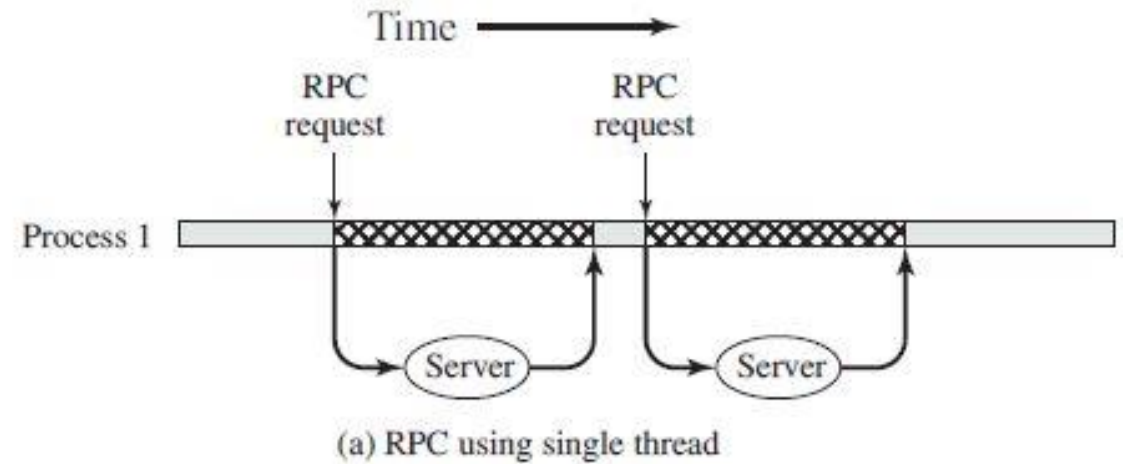
- ▨ Blocked, waiting for response to RPC
- Blocked, waiting for processor, which is in use by Thread B
- Running

**Figure 4.3** Remote Procedure Call (RPC) Using Threads

# The role of threads in RPC

## Multithreaded approach:

- ▶ Inside Process 1, Thread A starts executing. It makes the first RPC and blocks.
- ▶ Thread B starts executing when Thread A blocks.
- ▶ Thread B makes the second RPC and blocks.
- ▶ There is a period when both threads of Process 1 are blocked.



(b) RPC using one thread per server (on a uniprocessor)

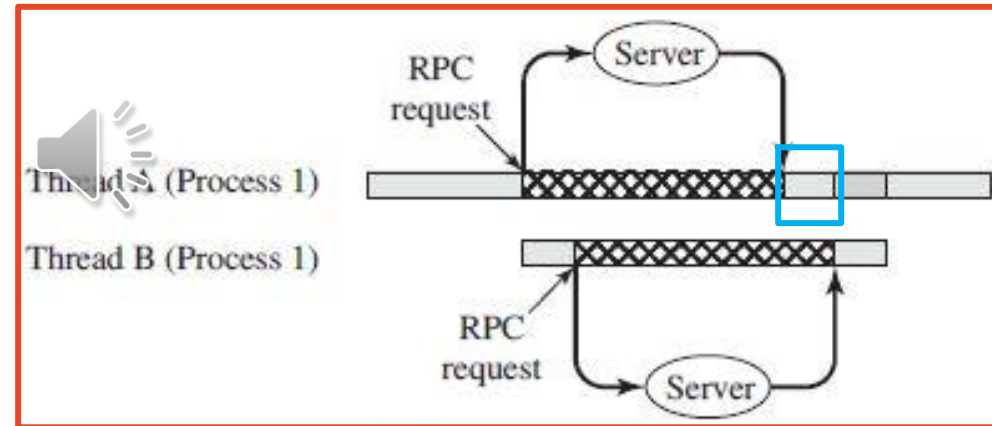
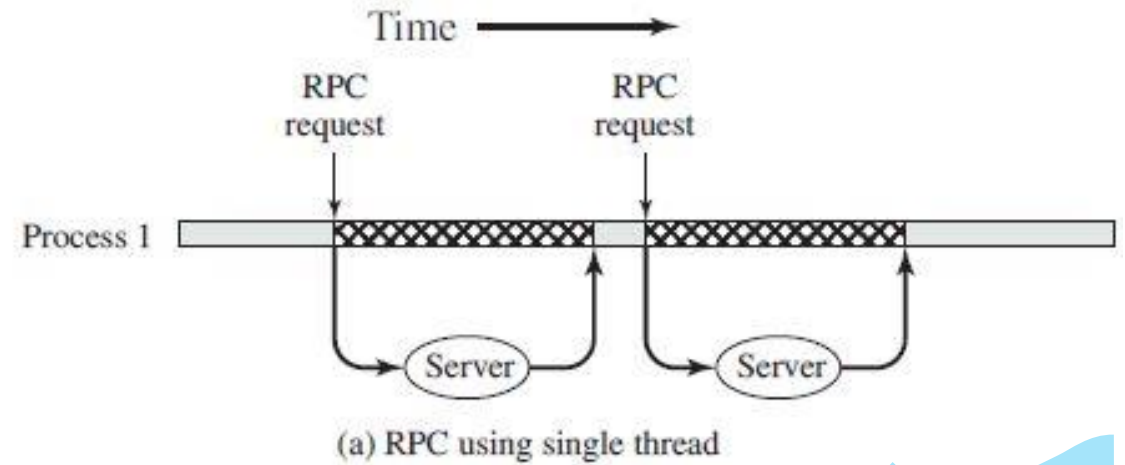
- Blocked, waiting for response to RPC
- Blocked, waiting for processor, which is in use by Thread B
- Running

**Figure 4.3** Remote Procedure Call (RPC) Using Threads

# The role of threads in RPC

## Multithreaded approach:

- ▶ Thread A starts executing when the server returns the result of the RPC.



(b) RPC using one thread per server (on a uniprocessor)

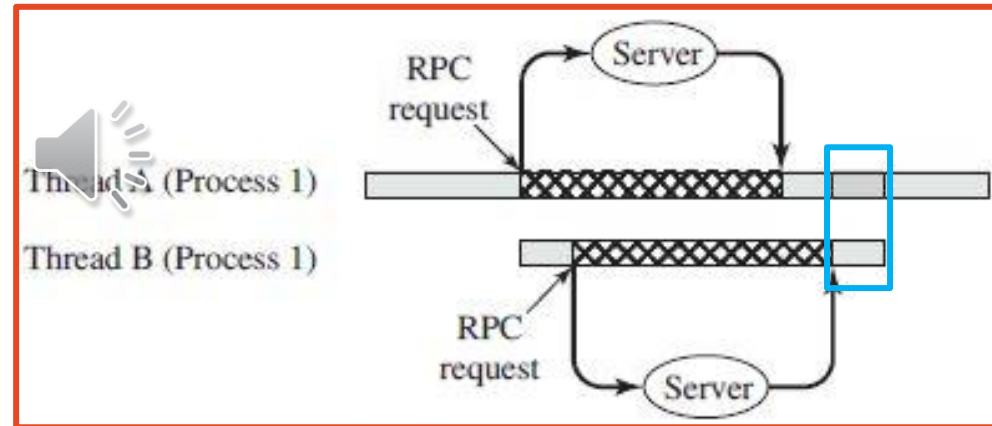
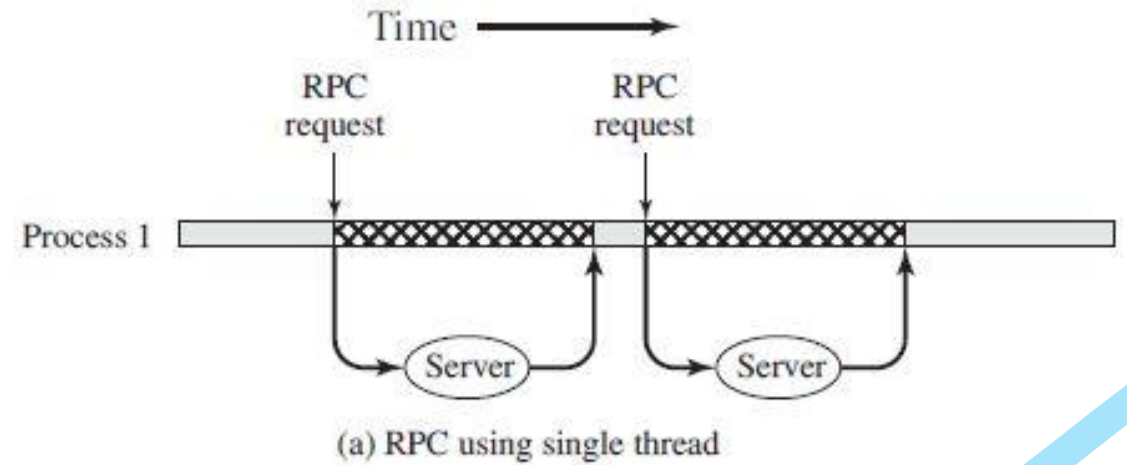
- Blocked, waiting for response to RPC
- Blocked, waiting for processor, which is in use by Thread B
- Running

**Figure 4.3** Remote Procedure Call (RPC) Using Threads

# The role of threads in RPC

## Multithreaded approach:

- ▶ Thread A starts executing when the server returns the result of the RPC.
- ▶ Thread B starts executing when the server returns the result of the RPC. Thread A is waiting for thread B to execute.



(b) RPC using one thread per server (on a uniprocessor)

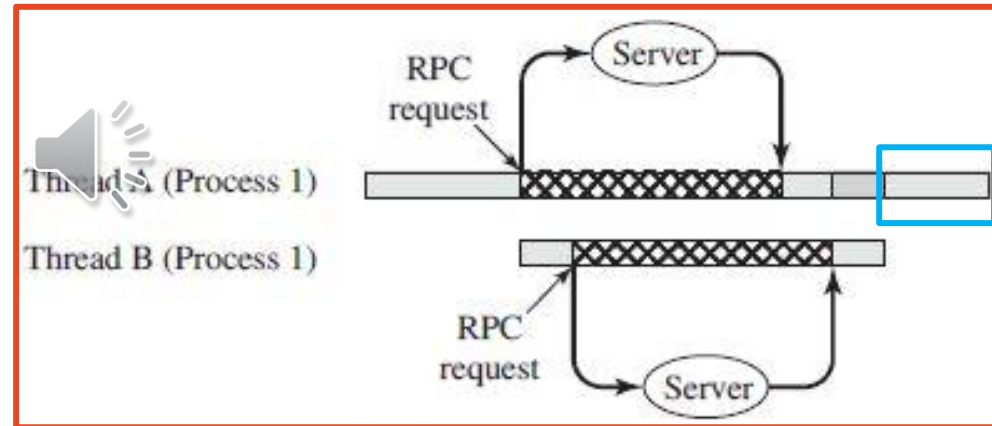
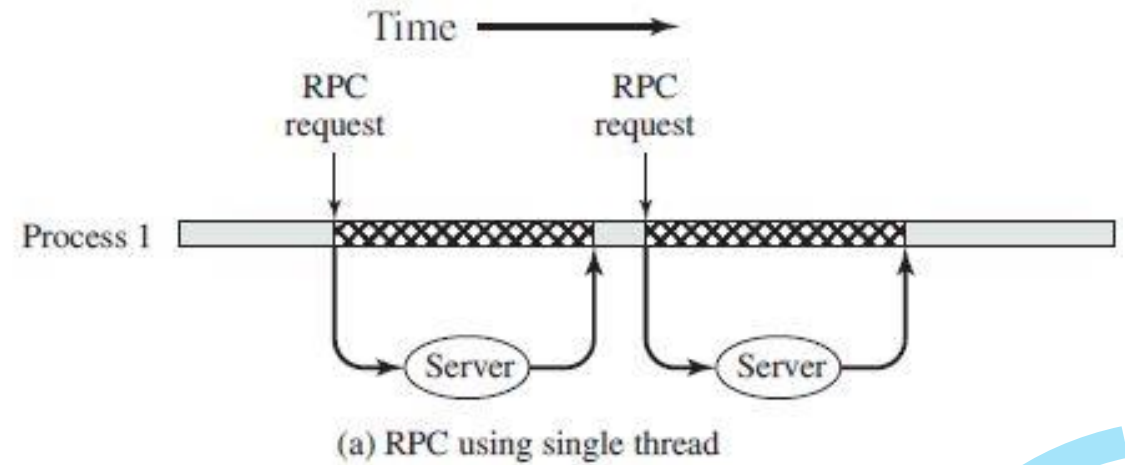
- Blocked, waiting for response to RPC
- Blocked, waiting for processor, which is in use by Thread B
- Running

**Figure 4.3** Remote Procedure Call (RPC) Using Threads

# The role of threads in RPC

## Multithreaded approach:

- ▶ Thread A starts executing when the server returns the result of the RPC.
- ▶ Thread B starts executing when the server returns the result of the RPC. Thread A is waiting for thread B to execute.
- ▶ Thread A regains processor and executes to completion.



(b) RPC using one thread per server (on a uniprocessor)

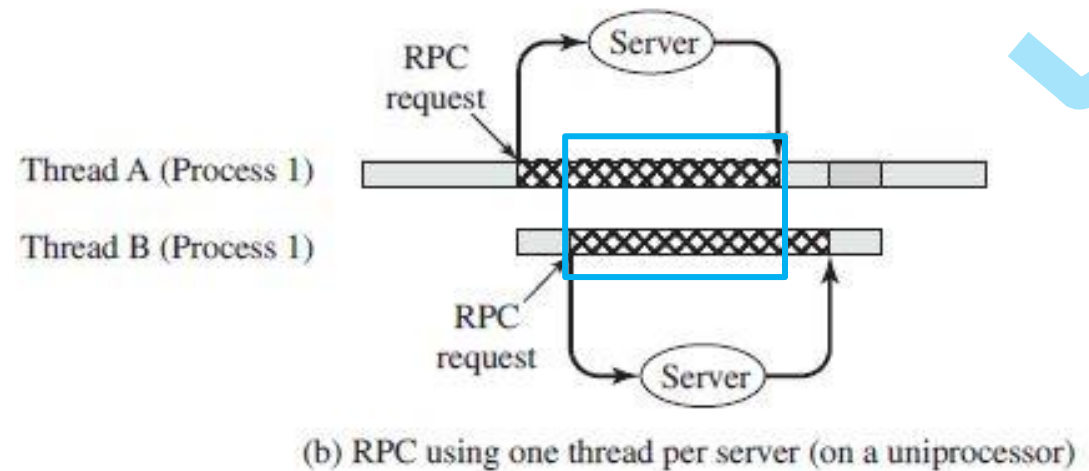
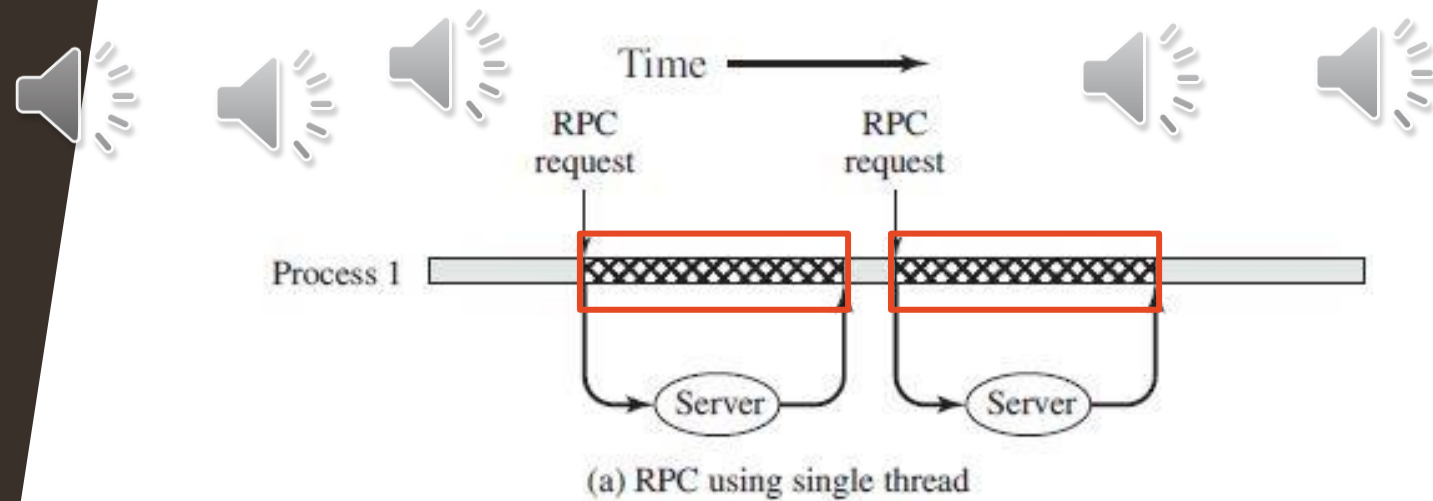
- ▨ Blocked, waiting for response to RPC
- Blocked, waiting for processor, which is in use by Thread B
- Running

**Figure 4.3** Remote Procedure Call (RPC) Using Threads

# The role of threads in RPC

## Idle time of the processor:

- ▶ We assume that Process 1 is the only process in the system.
- ▶ In the single-threaded approach, the idle time of the processor is highlighted by two red boxes.
- ▶ In the multithreaded approach the idle time of the processor is highlighted by a blue box.



- ▨ Blocked, waiting for response to RPC
- ▤ Blocked, waiting for processor, which is in use by Thread B
- ▬ Running

**Figure 4.3** Remote Procedure Call (RPC) Using Threads

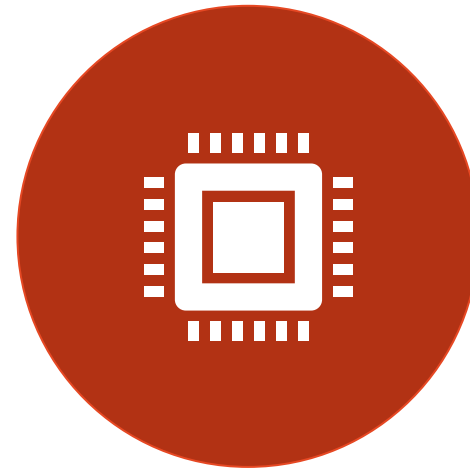
# Types of threads

Different ways of implementation of threads

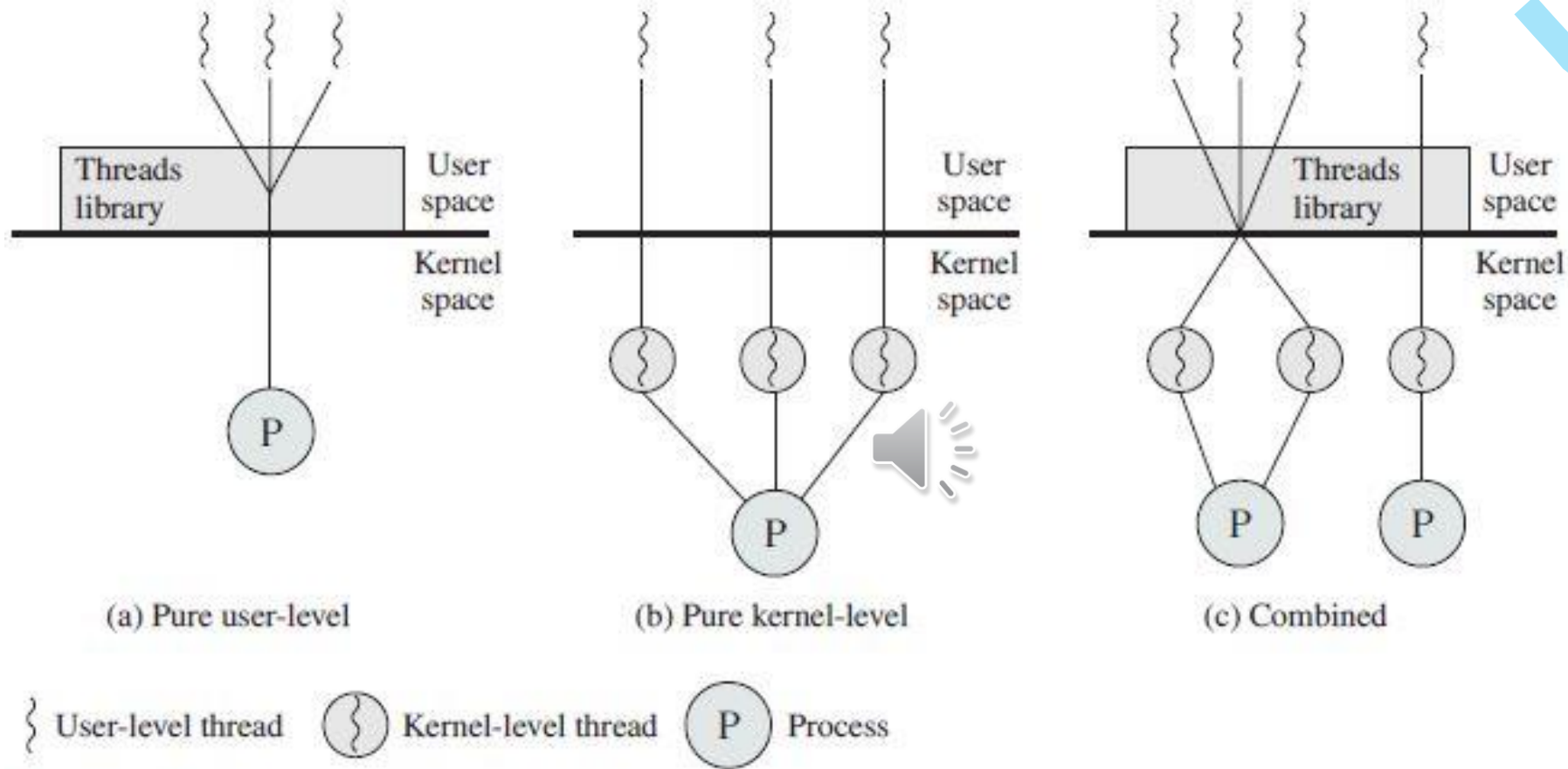
# Types of threads



USER LEVEL THREADS  
(ULTS)



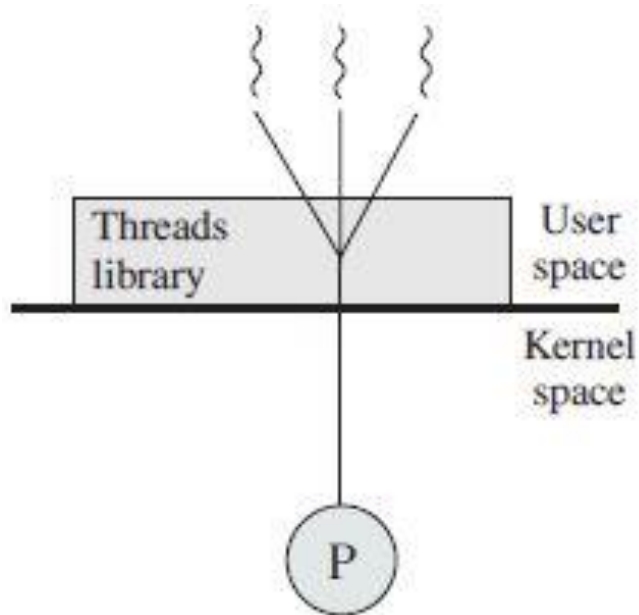
KERNEL LEVEL THREADS  
(KLTS)



**Figure 4.5** User-Level and Kernel-Level Threads

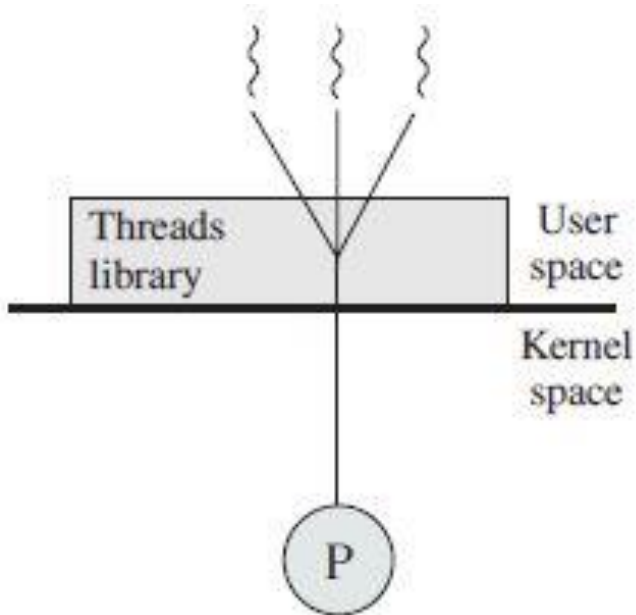
# Approaches to thread implementation

## User Level Threads



- In a pure ULT facility, all the work of thread management is done by the application and the kernel is not aware of the existence of threads.
- Any application can be programmed to be multithreaded by using a **threads library**, which is a package of routines for ULT management.
- The threads library contains code for:
  - creating and destroying threads,
  - passing messages and data between threads,
  - scheduling thread execution, and
  - saving and restoring thread contexts.
- By default, an application begins with a single thread and begins running in that thread.
- This application and its thread are allocated to a single process managed by the kernel.

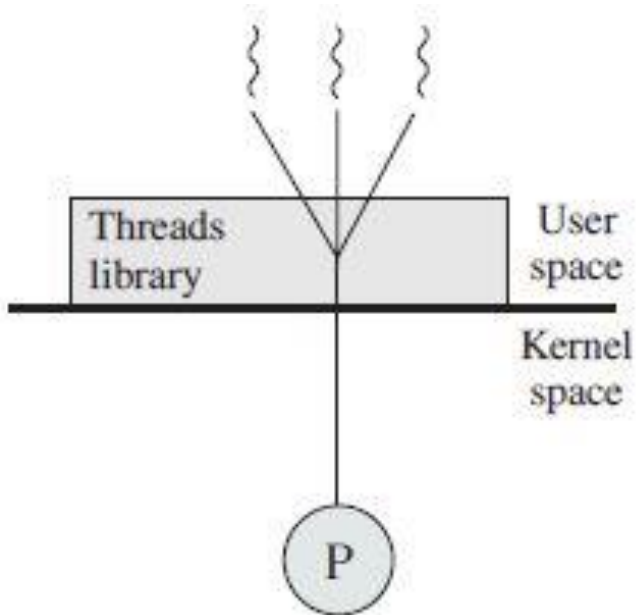
## User Level Threads



- At any time that the process is in the running state, the application may spawn a new thread to run within the same process.
- Spawning is done by invoking the spawn utility in the threads library.
- Control is passed to that utility by a procedure call.
- Threads library creates a TCB for the new thread and then passes control to one of the ready threads within this process using some scheduling algorithm.
- When control is passed from the thread to the library, the context of the current thread is saved.
- When control is passed from the library to a thread, the context of that thread is restored.

Yes — in user-level threads, a thread library (utility program) is responsible for dispatching the threads.

## User Level Threads

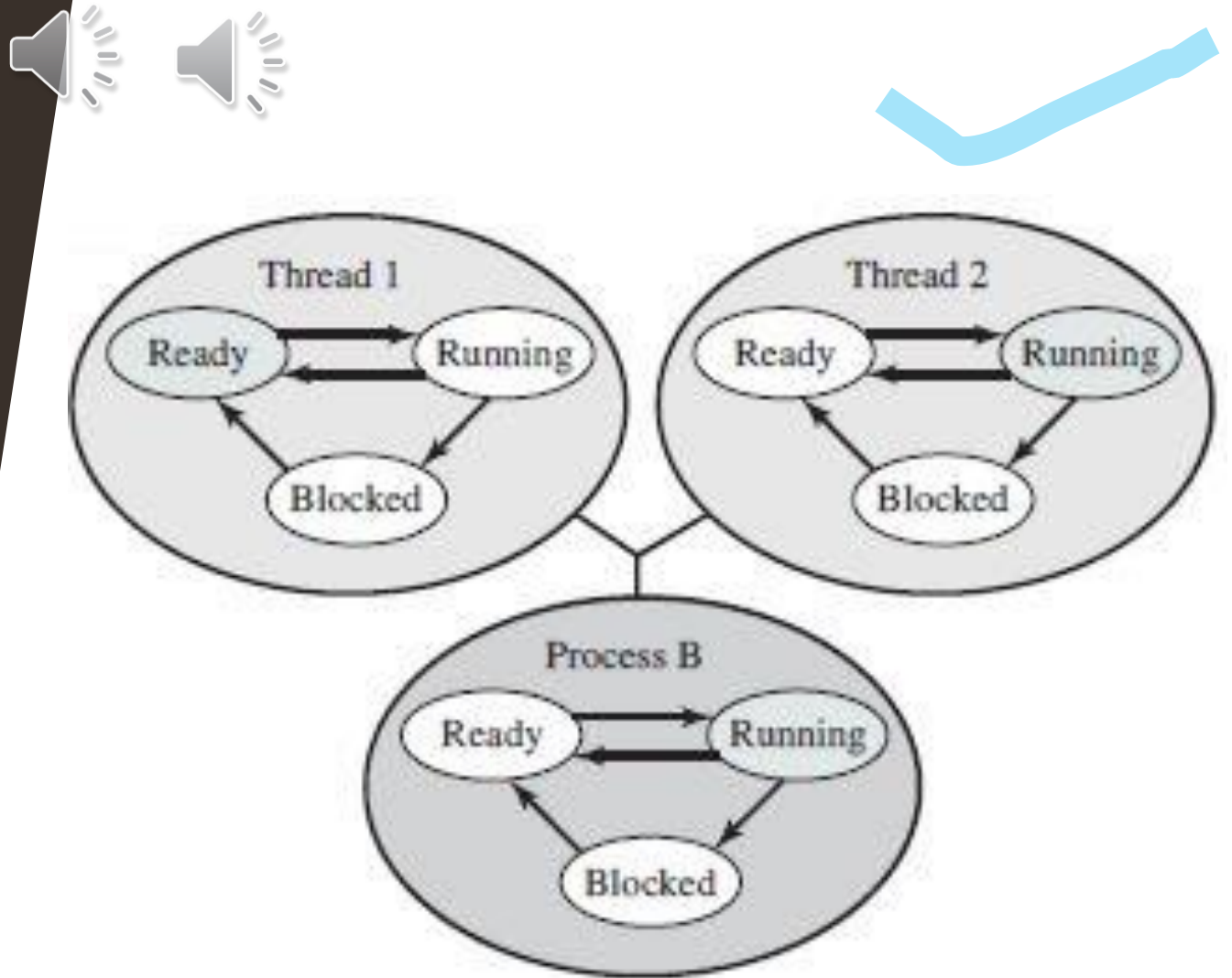


- All the activity of thread management takes place in user space and within a single process.
- The kernel is unaware of this activity.
- The kernel continues to schedule the process as a unit and assigns a single execution state (ready, running, blocked, etc.) to that process.
- For the OS, the threads created by the threads library do not exist. It cannot see them at all. It only recognizes the process.
- ULTs can run only when their containing process is chosen for execution by the OS.

# Possible scenarios with ULTs

## Starting situation:

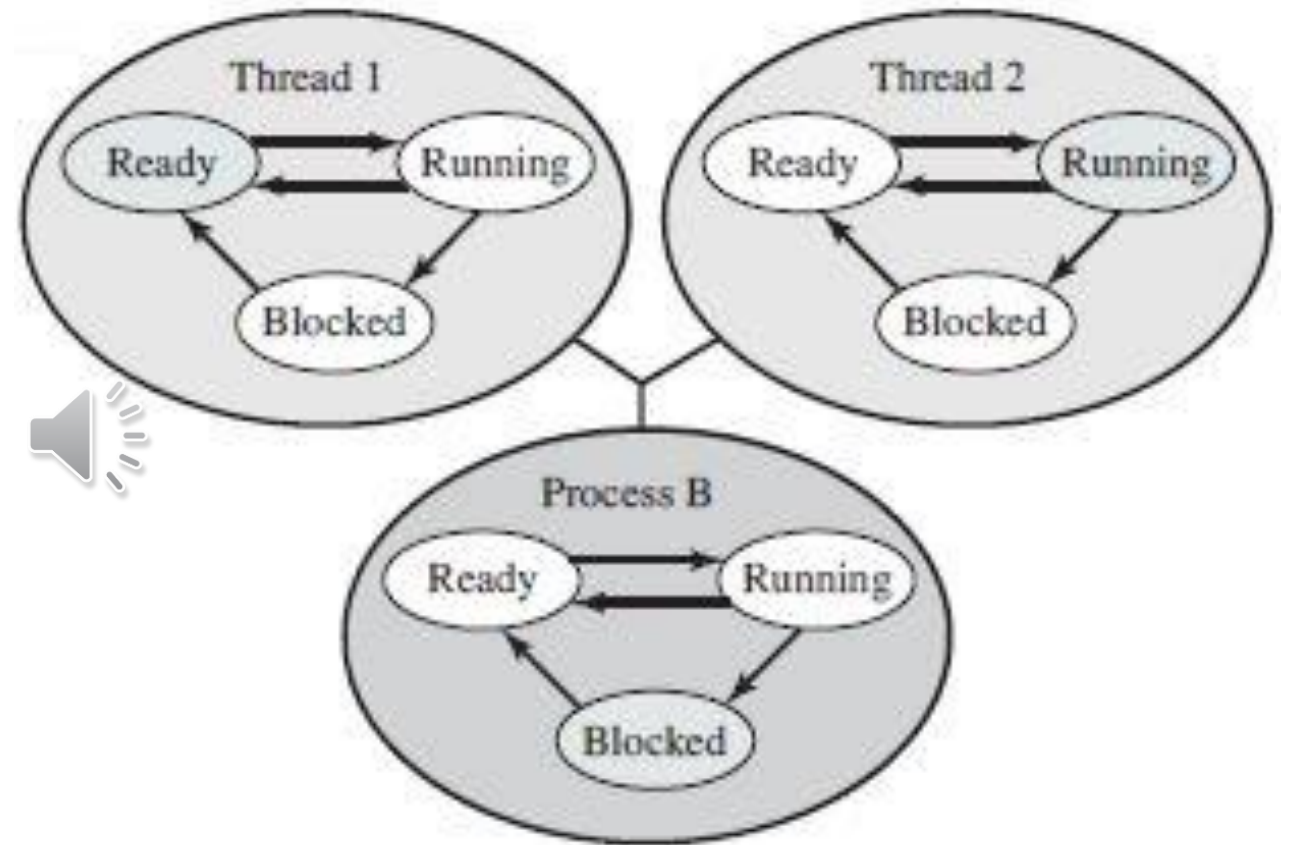
- ▶ Two ULTs, 1 and 2, are created inside process B by the threads library.
- ▶ Currently process B is running.
- ▶ In the user space, the threads library has scheduled thread 2 to run.
- ▶ Thread 1 is marked ready by the threads library.



# Possible scenarios with ULTs

Thread 2 makes a system call that blocks B:

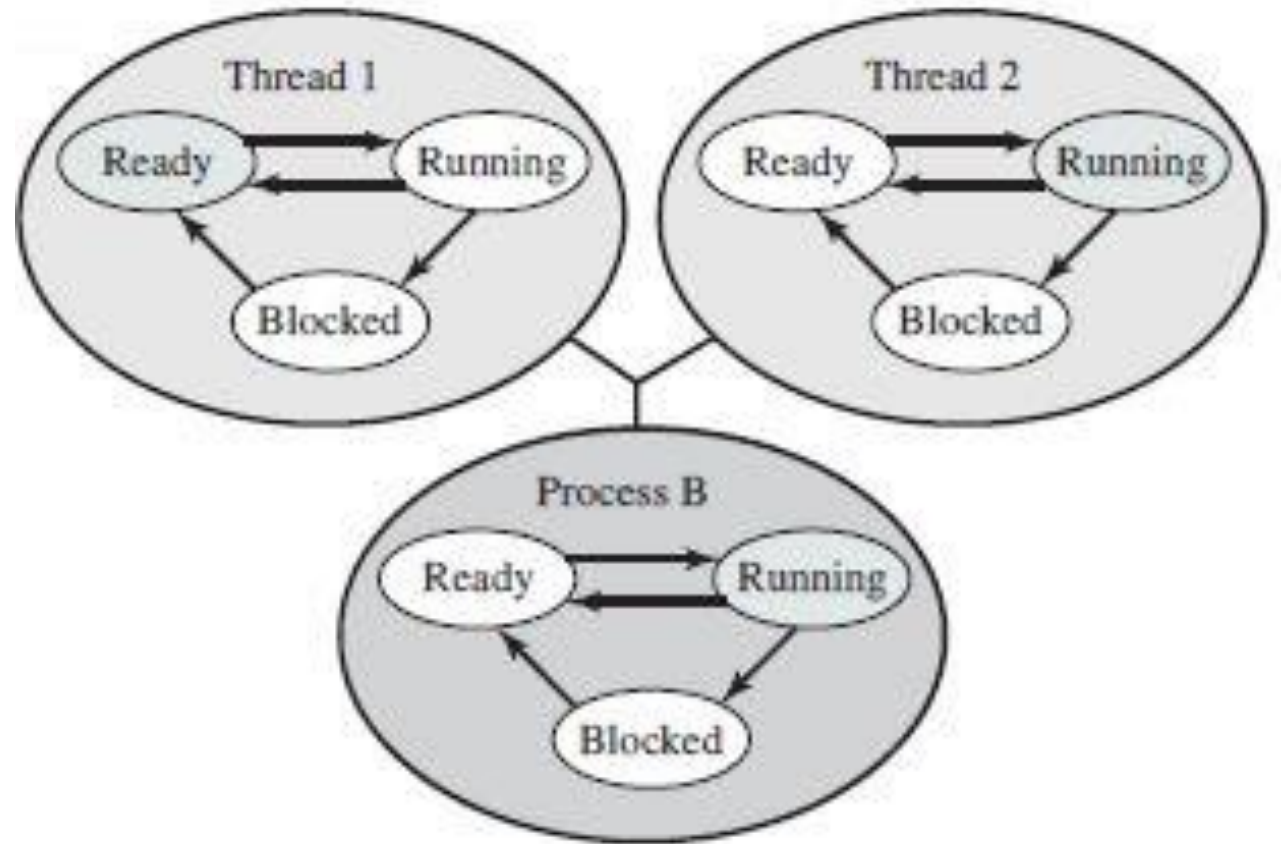
- ▶ The kernel invokes the I/O action, places process B in the blocked state, and switches to another process.
- ▶ According to the data structures maintained by the threads library, thread 2 of process B is still in the running state.



# Possible scenarios with ULTs

## Starting situation:

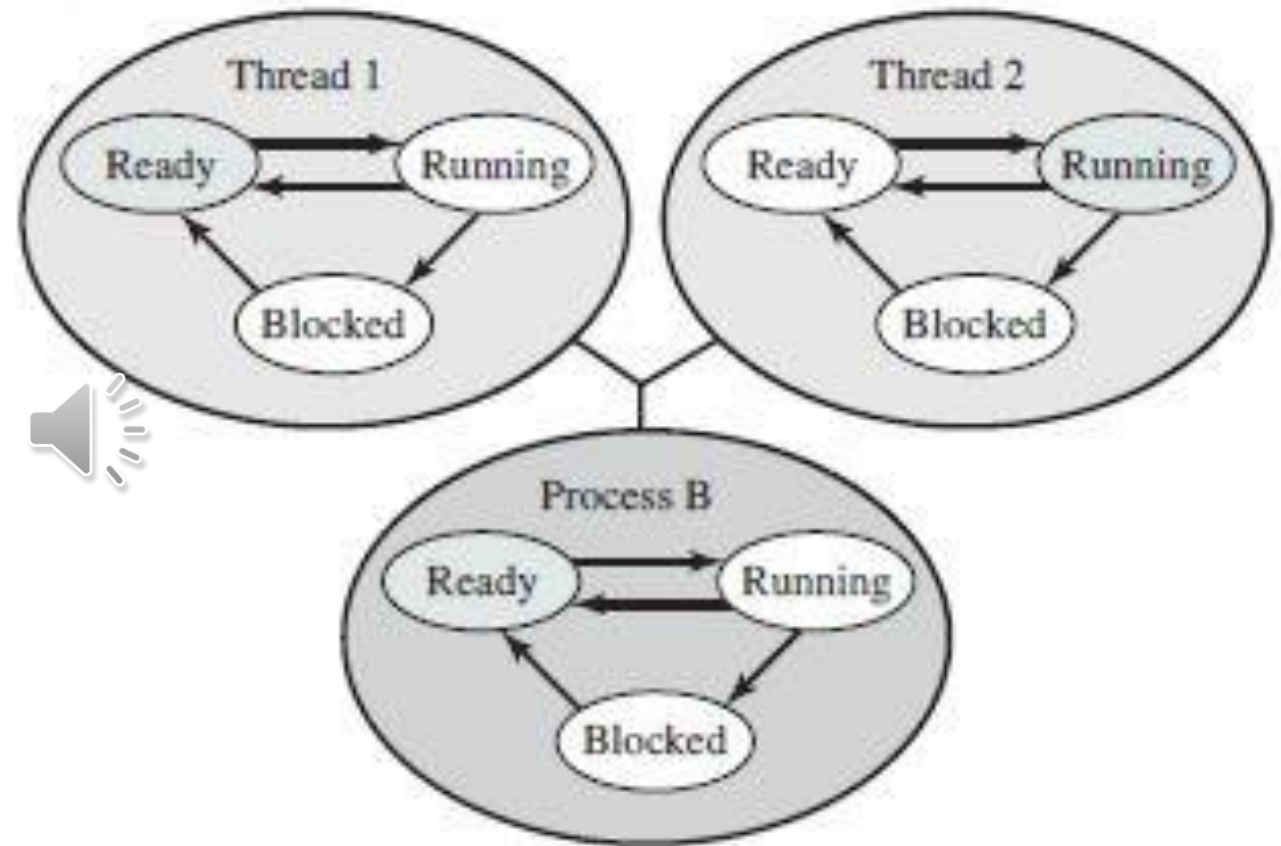
- ▶ Two ULTs, 1 and 2, are created inside process B by the threads library.
- ▶ Currently process B is running.
- ▶ In the user space, the threads library has scheduled thread 2 to run.
- ▶ Thread 1 is marked ready by the threads library.



# Possible scenarios with ULTs

Process B exhausts its time slice:

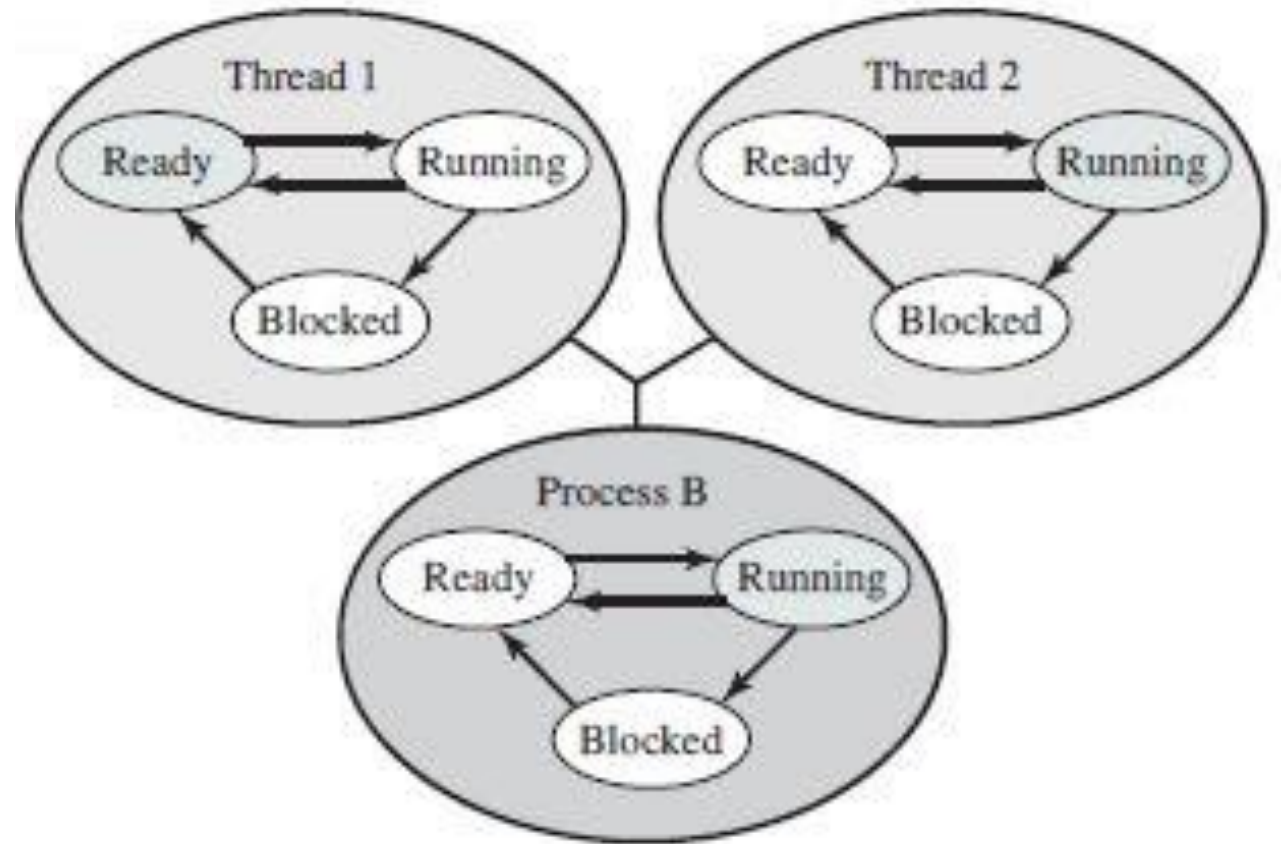
- ▶ The kernel places process B in the ready state and switches to another process.
- ▶ According to the data structure maintained by the threads library, thread 2 of process B is still in the running state.



# Possible scenarios with ULTs

## Starting situation:

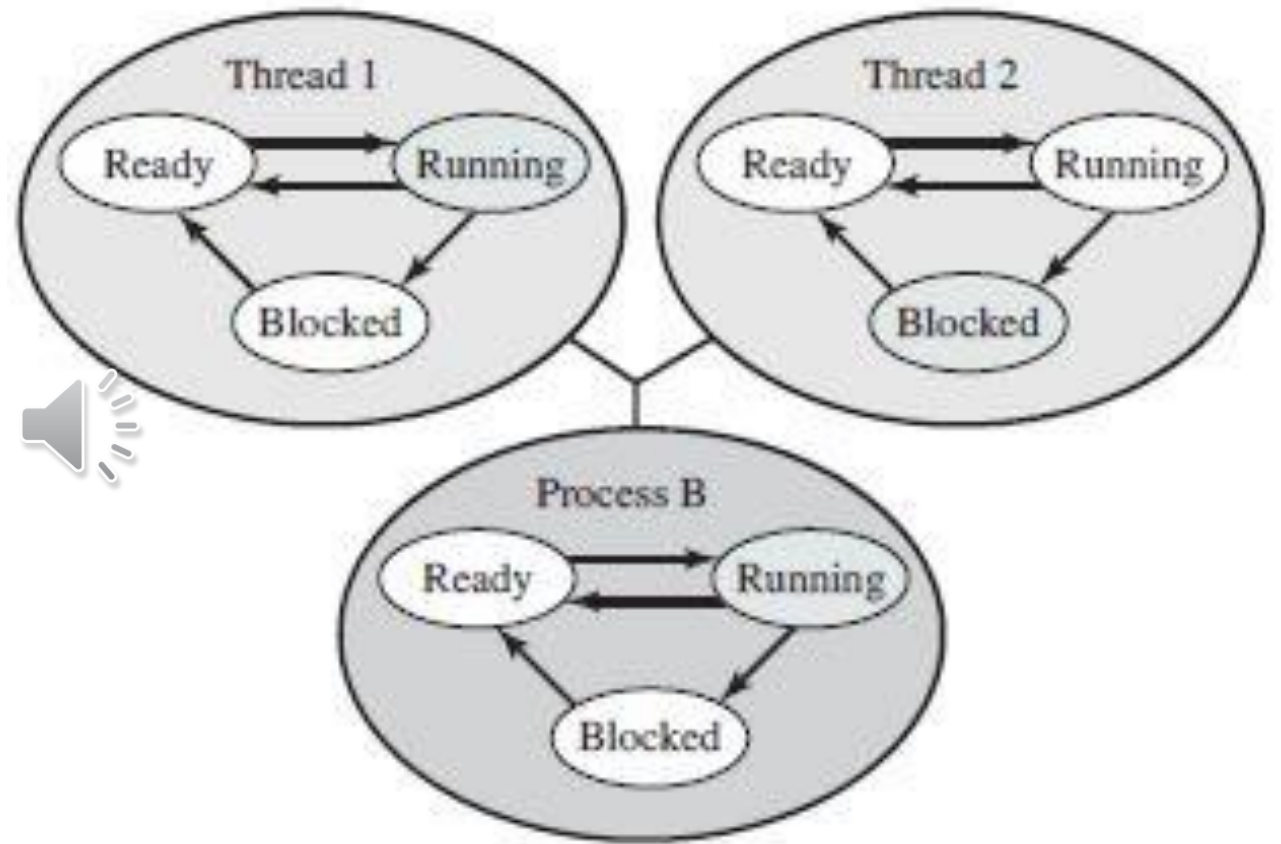
- ▶ Two ULTs, 1 and 2, are created inside process B by the threads library.
- ▶ Currently process B is running.
- ▶ In the user space, the threads library has scheduled thread 2 to run.
- ▶ Thread 1 is marked ready by the threads library.



# Possible scenarios with ULTs

Thread 2 needs some action performed by thread 1:

- ▶ Thread 2 enters the blocked state.
- ▶ Thread 1 transitions from ready to running.
- ▶ The process itself remains in the running state.
- ▶ The thread switch is invisible to the kernel.



# Interrupting a process containing ULTs

- ▶ A process can be interrupted, either by exhausting its time slice or by being preempted by a higher-priority process, while it is executing code in the threads library.
- ▶ A process may be in the midst of a thread switch from one thread to another when interrupted.
- ▶ When that process is resumed, execution continues within the threads library, which completes the thread switch and transfers control to another thread within that process.

# Advantages of ULTs

- ▶ Thread switching does not require kernel mode privileges because all the thread management data structures are within the user address space of a single process.
- ▶ Scheduling can be application specific. The scheduling algorithm in the threads library can be tailored to the application without disturbing the underlying OS scheduler.
- ▶ ULTs can run on any OS. No changes are required to the underlying kernel to support ULTs.

# Disadvantages of ULTs

- ▶ When a ULT executes a system call, not only is that thread blocked, but also all the threads within the process are blocked.
- ▶ In a pure ULT strategy, a multithreaded application cannot take advantage of multiprocessing. The kernel assigns one process to only one processor at a time. Therefore, only a single thread within a process can execute at a time.

## Work around for both disadvantages of ULTs

- ▶ Write an application as multiple processes rather than multiple threads.
- ▶ **Effect:** Each switch becomes a process switch rather than a thread switch, resulting in much greater overhead.

# Work around for disadvantage# 1 of ULTs

- ▶ **Disadvantage:** When a ULT executes a system call, not only is that thread blocked, but also all the threads within the process are blocked.
- ▶ **Solution:** Use jacketing.
- ▶ Jacketing converts a blocking system call into a nonblocking system call.

# Jacketing

- ▶ Instead of directly calling a system I/O routine, a thread calls an application-level I/O jacket routine.
- ▶ This jacket routine checks to determine if the I/O device is busy.
- ▶ If it is, the thread passes control to another thread.
- ▶ When this thread later is given control again, the jacket routine checks the I/O device again.

Miss ULT



Hello sir! I need a  
data transfer from  
I/O port 123.



That I/O device is busy.  
How dare you ask for it? I  
declare you blocked as long  
as the device is  
unavailable!



Sir OS

A woman with long red hair, wearing a black coat, red top, blue pants, and red boots, carrying a red bag.

Hello sir! I need a data transfer from I/O port 123. Can you check?

Miss ULT



Mr. Jacket

An older man with white hair, wearing a blue suit, yellow shirt, and red tie, standing next to a safe.

That I/O device is busy. I just checked the I/O tables. Don't say a word. Come back later. Ill check again.



Sir OS

# Lessons learned

- ▶ We looked at thread states.
- ▶ We were introduced to the idea of synchronization and its relation to threads.
- ▶ We realized how RPCs can benefit from multithreading.
- ▶ We explored user level threads.

# Things to do



Read the book.



I will ask you to submit the word file previously saved as <my\_roll\_no>\_lecture\_8 in Google classroom.



Write your questions on a piece of paper and keep it in front of you during the online session.



If you have suggestions relevant to content or quality of this lecture, email them to me.